
Predicción de Consumo de Combustible (MPG)

Proyecto Modulo 5 — Aprendizaje Supervisado

Auto MPG Dataset · UCI Machine Learning Repository · 398 observaciones

Integrantes del equipo

Alan A. Castañeda Ramos · Daniel Macias Soto · Guillermo
A. Martinez Vanegas · Mariana Vigil Villegas · Ronaldo
Berrocal Reyes

Curso: Introducción Analítica a la Ciencia de Datos

27 de junio de 2026

Índice

1. Resumen Ejecutivo	2
2. Descripción del Problema y los Datos	2
2.1. Contexto	2
2.2. Fuente de datos	2
2.3. Descripción de las variables	3
3. Análisis Exploratorio de Datos (EDA)	3
3.1. Revisión inicial	3
3.2. Distribución de las variables	4
3.3. Análisis de correlaciones	5
3.4. Relaciones multivariadas	6
3.5. Detección de valores atípicos	7
4. Preprocesamiento	7
4.1. Tratamiento de valores nulos	7
4.2. Codificación de variables categóricas	8
4.3. División del conjunto de datos	8
4.4. Escalado de variables	9
4.5. Resumen del pipeline de preprocesamiento	10
5. Modelado Mediante Aprendizaje Supervisado	10
5.1. Modelos Lineales y Regularización	10
5.1.1. Comparación de desempeño	11
5.1.2. Diagnóstico del modelo	13
5.1.3. Interpretación de los coeficientes	13
5.2. Random Forest	15
5.3. Random Forest mediante GridSearch	17
5.4. AdaBoost	20
5.5. Ada Boost mediante GridSearch	21
5.6. XGBoost	23
5.7. XGBoost mediante GridSearch	25
5.8. Red Neuronal — Perceptrón Multicapa (MLP)	27
5.8.1. Optimización de funciones de pérdida: MSE vs MAE	29
5.8.2. Validación cruzada 10-fold	30
5.9. Comparativa Final de Modelos	30
5.9.1. Explicabilidad: Lasso vs Random Forest	31
5.10. Conclusiones	33

1 Resumen Ejecutivo

Resumen Ejecutivo

Este proyecto entrena, evalúa y compara modelos de aprendizaje supervisado para predecir el consumo de combustible de automóviles medido en millas por galón (MPG) a partir de 7 características técnicas del vehículo.

Dataset: Auto MPG (UCI), 398 automóviles fabricados entre 1970 y 1982.

Hallazgo principal: El peso del vehículo (**weight**), la cilindrada (**displacement**) y el número de cilindros (**cylinders**) son los predictores más determinantes del consumo. Las tres presentan multicolinealidad severa ($r > 0,85$) al medir indirectamente el tamaño del motor, lo que motivó el uso de regularización Ridge y Lasso como técnicas de control y selección de variables respectivamente.

Resultado: Se entrenaron 12 modelos agrupados en cuatro familias: lineales, árbol de decisión, ensambles y red neuronal. Los modelos de ensamble que dominaron la comparativa fueron **Random Forest Optimizado** lideró con $R^2 = 0,9156$ y MAE = 1,55 MPG, mientras que el baseline lineal (OLS) explicó el 84.5 % de la varianza con un MAE de 2.29 MPG, una diferencia de 7 puntos porcentuales. Lasso aportó interpretabilidad al eliminar automáticamente **cylinders** por redundancia con **weight** y **displacement**, confirmando la multicolinealidad detectada en el EDA.

2 Descripción del Problema y los Datos

2.1 Contexto

La optimización del rendimiento de combustible en vehículos con motores de combustión interna representa un problema fundamental dentro del sector automotriz. Este proyecto trata de predecir el rendimiento en millas por galón (MPG) de diversos automóviles a partir de sus características técnicas.

2.2 Fuente de datos

El *Auto MPG Dataset* fue obtenido del repositorio UCI Machine Learning [1] mediante la librería oficial `ucimlrepo`:

```
1 from ucimlrepo import fetch_ucirepo
2
3 auto_mpg = fetch_ucirepo(id=9)
4 X = auto_mpg.data.features      # 7 variables predictoras
5 y = auto_mpg.data.targets      # variable objetivo: mpg
```

Listing 1: Carga del dataset desde UCI

2.3 Descripción de las variables

Cuadro 1: Variables del dataset Auto MPG

Variable	Tipo	Descripción	Rol
mpg	Continua	Millas por galón (consumo)	Target
cylinders	Entera	Número de cilindros (3–8)	Feature
displacement	Continua	Desplazamiento del motor (pulg ³)	Feature
horsepower	Continua	Potencia en caballos de fuerza	Feature
weight	Continua	Peso del vehículo (libras)	Feature
acceleration	Continua	Tiempo de 0 a 60 mph (segundos)	Feature
model_year	Entera	Año de fabricación (70–82)	Feature
origin	Categorica	Región de origen (USA/Europa/-Japón)	Feature
car_name	Texto	Nombre del automóvil	ID (excluido)

3 Análisis Exploratorio de Datos (EDA)

3.1 Revisión inicial

Resumen Ejecutivo

El dataset contiene **398 observaciones y 8 variables** (7 predictoras + 1 objetivo). Se identificó únicamente un problema de calidad: 6 valores nulos en la variable **horsepower**, representando el 1.5 % de los datos. El resto del dataset está completo.

Detalle Técnico

Estadísticas descriptivas clave:

Cuadro 2: Estadísticas descriptivas del dataset

Variable	Media	Desv. Est.	Mín.	Mediana	Máx.
mpg	23.45	7.81	9.0	23.0	46.6
cylinders	5.47	1.70	3	4	8
displacement	194.4	104.6	68.0	151.0	455.0
horsepower	104.5	38.5	46.0	93.5	230.0
weight	2977	849	1613	2803	5140
acceleration	15.54	2.76	8.0	15.5	24.8
model_year	75.98	3.70	70	76	82

Variable categórica origin — distribución de frecuencias:

Cuadro 3: Distribución de la variable categórica `origin`

Origen	Frecuencia	Porcentaje
1 — USA	245	61.6 %
2 — Europa	68	17.1 %
3 — Japón	79	19.8 %
Total	398	100 %

3.2 Distribución de las variables

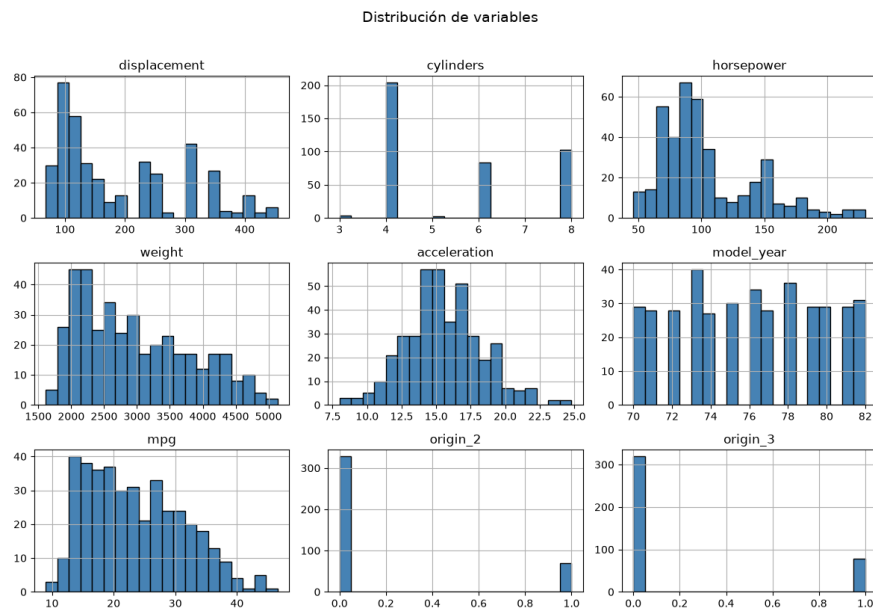


Figura 1: Distribución de todas las variables del dataset Auto MPG.

- `mpg` presenta un sesgo positivo, tal que, la mayoría de automóviles rinde entre 15 y 30 MPG, con pocos autos por encima de 35 MPG.
- `cylinders` solo toma valores discretos (3, 4, 5, 6, 8), comportándose como variable ordinal. El 51 % de los autos tiene 4 cilindros.
- `displacement`, `horsepower` y `weight` muestran distribuciones asimétricas con colas largas hacia la derecha, anticipando la presencia de valores atípicos influyentes.

3.3 Análisis de correlaciones

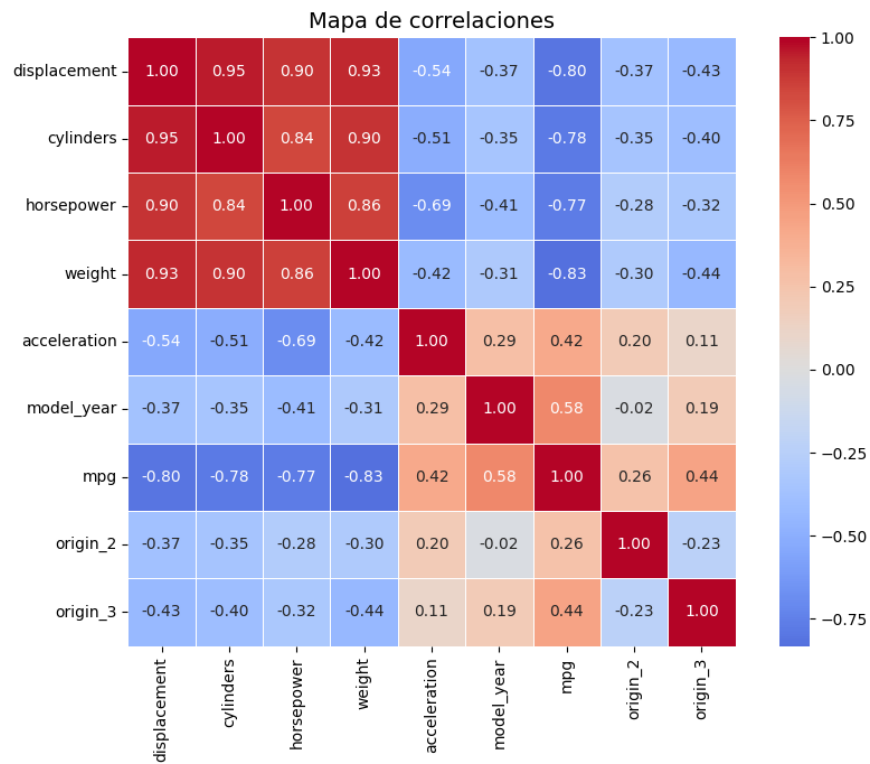


Figura 2: Mapa de correlaciones

Multicolinealidad severa: displacement, cylinders y weight presentan correlaciones entre sí superiores a 0.85 (fig. 2), dado que las tres variables miden indirectamente el tamaño del motor.

Detalle Técnico

Correlaciones con la variable objetivo mpg:

Cuadro 4: Correlación de Pearson de cada variable con mpg

Variable	Correlación	Interpretación
weight	-0,83	Más peso = menos eficiencia
displacement	-0,80	Más motor = menos eficiencia
cylinders	-0,78	Más cilindros = menos eficiencia
horsepower	-0,77	Más potencia = menos eficiencia
model_year	+0,58	Autos más recientes = más eficientes
origin ₂	+0,26	
origin ₃	+0,44	
acceleration	+0,42	Variable menos predictiva del conjunto

3.4 Relaciones multivariadas

Los diagramas de dispersión (fig. 3) confirman que la relación entre **weight**, **displacement** y **mpg** no es perfectamente lineal, lo que anticipa que los modelos no lineales (Random Forest, XGBoost) podrán capturar mejor estos patrones que los modelos lineales.

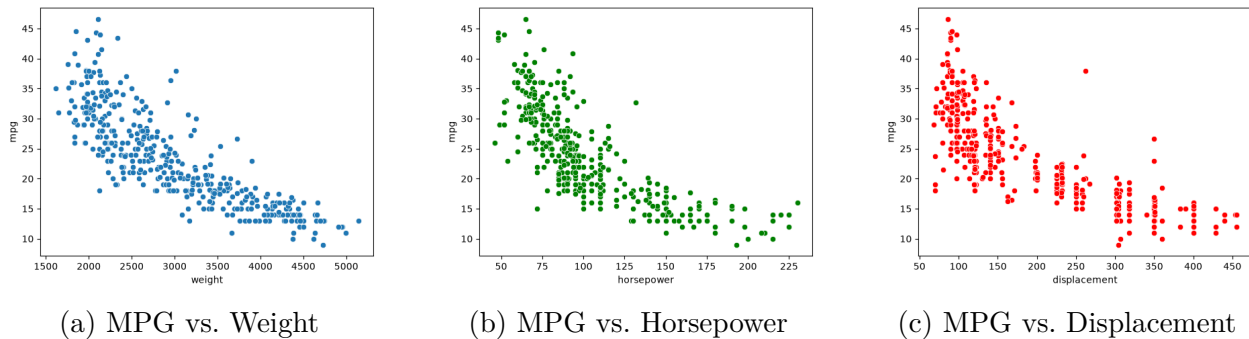


Figura 3: Relaciones bivariadas entre las variables más correlacionadas con MPG.

3.5 Detección de valores atípicos

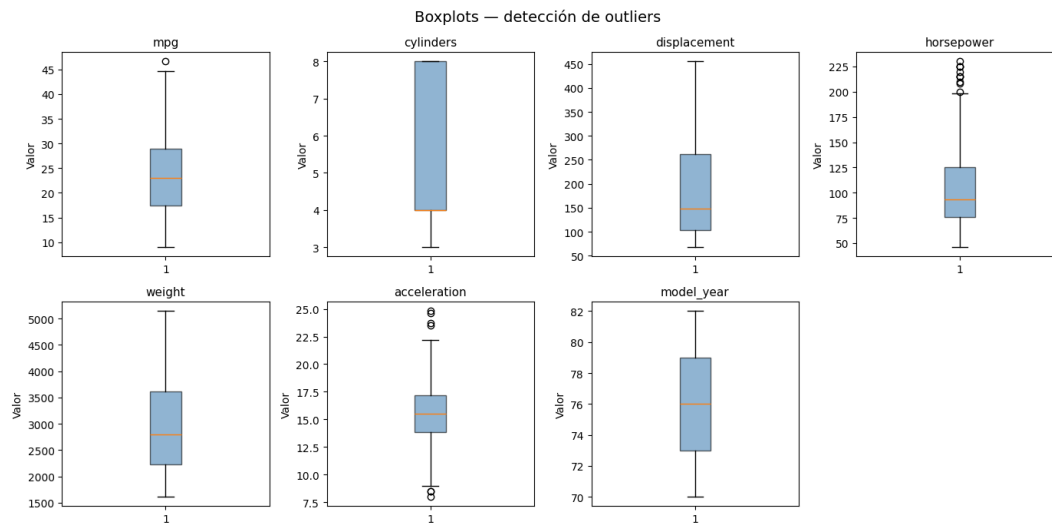


Figura 4: Boxplots para detección de valores atípicos en todas las variables.

Outliers: horsepower y acceleration presentan valores atípicos visibles fuera de los bigotes superiores. mpg también muestra valores atípicos en el rango superior (autos muy eficientes).

4 Preprocesamiento

4.1 Tratamiento de valores nulos

Resumen Ejecutivo

Se identificaron **6 valores nulos** en la variable **horsepower** (1.5 % del dataset). Fueron imputados con la **mediana** en lugar de la media, dado el sesgo positivo de la distribución.

Detalle Técnico

Justificación técnica de la imputación por mediana:

En este dataset: $\mu \approx 104,5$ HP vs. $\tilde{x} = 93,5$ HP, una diferencia de 11 HP causada por los **13 automóviles con más de 150 HP** que sesgan la distribución hacia arriba. Imputar con μ le asignaría a los 6 autos desconocidos más potencia de la representativa, distorsionando el modelo.

```
1 mediana = df['horsepower'].median()    # 93.5 HP
2 df['horsepower'] = df['horsepower'].fillna(mediana)
```


Listing 2: Imputación con mediana

4.2 Codificación de variables categóricas

Resumen Ejecutivo

La variable `origin` fue convertida a dos variables dummy mediante *one-hot encoding*, usando USA (`origin=1`) como categoría de referencia.

Detalle Técnico

`origin` contiene tres categorías: 1=USA (61.6 %), 2=Europa (17.1 %) y 3=Japón (19.8 %). Se aplicó *one-hot encoding* con `drop_first=True`, generando `origin_2` (Europa) y `origin_3` (Japón):

Eliminar la primera categoría (`drop_first=True`) evita la **trampa de la variable dummy**: si se incluyeran las tres columnas, su suma siempre valdría 1, introduciendo multicolinealidad perfecta que hace indeterminado el sistema de ecuaciones normales de los modelos lineales.

4.3 División del conjunto de datos

Resumen Ejecutivo

El dataset se dividió en **80 % entrenamiento (318 observaciones)** y **20 % prueba (80 observaciones)**, fijando `random_state=42` para garantizar la reproducibilidad.

Detalle Técnico

Dado el tamaño reducido del dataset (398 observaciones), una división 80/20 maximiza los datos de entrenamiento manteniendo un conjunto de prueba representativo.

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X, y, test_size=0.20, random_state=42  
3 )  
4 # Train: (318, 8) / Test: (80, 8)
```

Listing 3: División train/test

4.4 Escalado de variables

Resumen Ejecutivo

Se aplicó `StandardScaler` para centrar todas las variables continuas en media 0 y desviación estándar 1. Las variables dummy `origin_2` y `origin_3` **no fueron escaladas** por estar ya en escala binaria (0/1). Aplicarles `StandardScaler` las convertiría en valores como $-0,53$ y $1,87$, sin interpretación lógica para una variable categórica codificada.

Detalle Técnico

La decisión de escalar o no cada modelo se resume en la siguiente tabla. Se mantienen dos versiones del dataset: `train_scaled.csv` para modelos sensibles a la escala y `train_raw.csv` para modelos basados en árboles, garantizando consistencia en la comparativa final.

Cuadro 5: Requerimiento de escalado por modelo

Modelo	Escalado	Razón
OLS, Ridge, Lasso, Elastic Net	Sí	La penalización L_1/L_2 es sensible a la escala de los coeficientes
MLP	Sí	El descenso de gradiente converge más rápido con variables normalizadas
Random Forest, AdaBoost, XGBoost	No	Usan divisiones binarias sobre los valores originales, no distancias ni penalizaciones

4.5 Resumen del pipeline de preprocesamiento

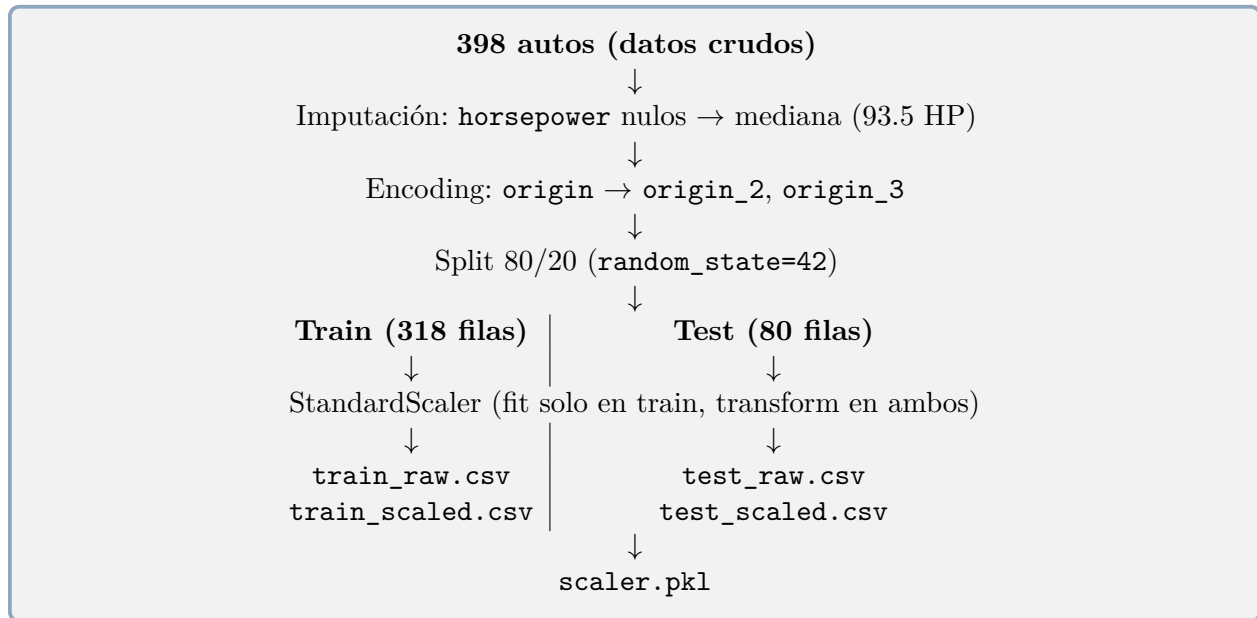


Figura 5: Pipeline completo de preprocesamiento.

5 Modelado Mediante Aprendizaje Supervisado

5.1 Modelos Lineales y Regularización

Resumen Ejecutivo

Como línea base del proyecto se implementaron modelos de regresión lineal con el objetivo de establecer un punto de referencia para comparar posteriormente el desempeño de modelos de mayor complejidad, tales como Random Forest, AdaBoost, XGBoost y Redes Neuronales.

Además de la regresión lineal ordinaria (OLS), se entrenaron tres variantes regularizadas: Ridge (L_2), LASSO (L_1) y Elastic Net. Estas técnicas permiten disminuir el efecto de la multicolinealidad identificada durante el análisis exploratorio y evaluar si la incorporación de penalizaciones mejora la capacidad predictiva del modelo.

Los modelos fueron entrenados utilizando el conjunto de entrenamiento descrito en la Sección 4.3. Para Ridge, LASSO y Elastic Net el parámetro de regularización fue seleccionado automáticamente mediante validación cruzada, mientras que todas las variables continuas fueron previamente estandarizadas mediante **StandardScaler**.

Detalle Técnico

La regresión lineal ordinaria (Ordinary Least Squares, OLS) estima los coeficientes del modelo minimizando la suma de los errores cuadráticos entre los valores observados y las predicciones realizadas. Este modelo constituye el punto de comparación natural para evaluar el beneficio de incorporar técnicas de regularización.

Las técnicas Ridge y LASSO modifican la función objetivo añadiendo una penalización sobre la magnitud de los coeficientes. Mientras Ridge incorpora una penalización cuadrática (L_2) que reduce la magnitud de todos los coeficientes sin eliminarlos completamente, LASSO utiliza una penalización absoluta (L_1), permitiendo llevar algunos coeficientes exactamente a cero y realizando de manera implícita selección de variables.

Elastic Net combina simultáneamente ambas penalizaciones, buscando equilibrar la estabilidad proporcionada por Ridge con la capacidad de selección automática de variables característica de LASSO.

Para seleccionar automáticamente el nivel óptimo de regularización se emplearon los algoritmos `RidgeCV`, `LassoCV` y `ElasticNetCV`, los cuales determinan el mejor parámetro mediante validación cruzada sobre el conjunto de entrenamiento.

5.1.1 Comparación de desempeño

Las predicciones obtenidas para los cuatro modelos fueron evaluadas mediante tres métricas ampliamente utilizadas en problemas de regresión: Error Absoluto Medio (MAE), Raíz del Error Cuadrático Medio (RMSE) y coeficiente de determinación (R^2). Mientras que MAE mide el error promedio absoluto de las predicciones, RMSE penaliza con mayor intensidad los errores grandes y R^2 cuantifica la proporción de variabilidad explicada por el modelo.

La Tabla 6 resume los resultados obtenidos sobre el conjunto de prueba.

Cuadro 6: Comparación del desempeño de los modelos lineales.

Modelo	MAE	RMSE	R^2
Regresión Lineal (OLS)	2.2882	2.8877	0.8449
Ridge	2.2895	2.8914	0.8445
LASSO	2.2935	2.9085	0.8427
Elastic Net	2.2961	2.9037	0.8432

Se observa que los cuatro modelos presentan un comportamiento muy similar. La regresión lineal ordinaria obtuvo los mejores valores en las tres métricas de evaluación, alcanzando un coeficiente de determinación cercano al 85 %, lo que indica que el modelo es capaz de explicar una proporción importante de la variabilidad presente en el consumo de combustible.

Aunque las técnicas de regularización reducen la complejidad del modelo y ayudan a controlar la multicolinealidad, en este conjunto de datos no produjeron mejoras apreciables en el desempeño predictivo. Esto sugiere que la relación entre las variables predictoras y el rendimiento de combustible puede describirse adecuadamente mediante un modelo lineal sin necesidad de introducir penalizaciones adicionales.

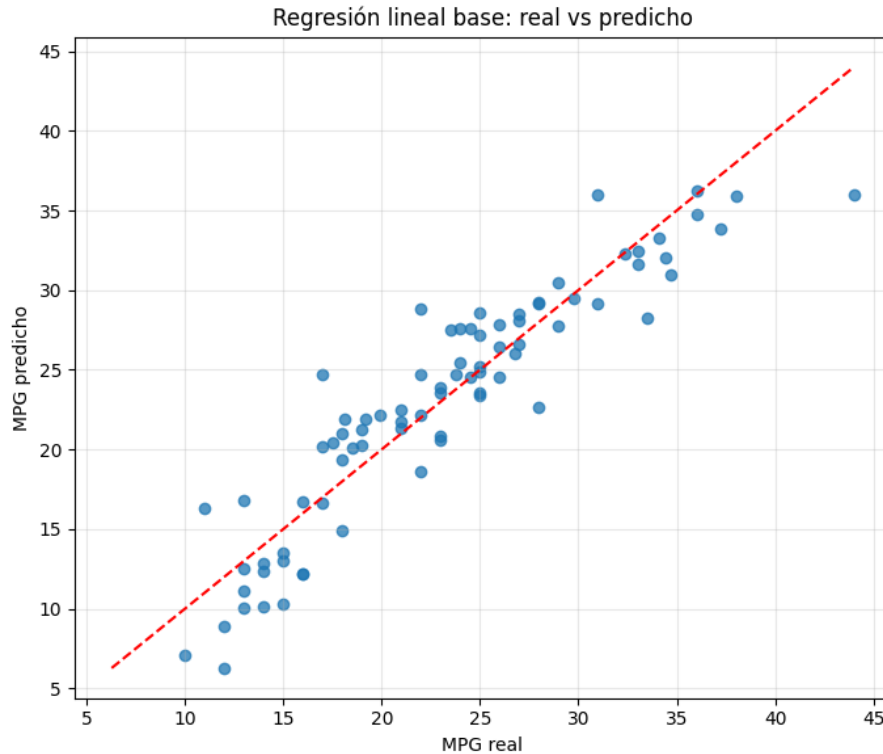


Figura 6: Comparación entre los valores reales y predichos obtenidos mediante regresión lineal ordinaria. La línea punteada representa el caso ideal donde la predicción coincide exactamente con el valor observado.

La Figura 6 muestra que la mayoría de las observaciones se concentran alrededor de la recta identidad, lo cual indica un buen ajuste general del modelo sobre el conjunto de prueba. Conforme aumenta el valor de MPG puede observarse una ligera tendencia del modelo a subestimar algunos de los vehículos con mayor eficiencia de combustible, comportamiento esperado debido a la menor cantidad de observaciones disponibles en ese rango del conjunto de datos.

A pesar de ello, la dispersión observada alrededor de la recta permanece relativamente pequeña, respaldando las métricas obtenidas y confirmando que la regresión lineal constituye un punto de partida sólido para comparar modelos de mayor complejidad.

La incorporación de penalizaciones mediante Ridge, LASSO y Elastic Net no produjo mejoras significativas respecto a la regresión lineal ordinaria. Este resultado indica que, aunque existen variables altamente correlacionadas, el nivel de sobreajuste del problema es reducido y un modelo lineal simple resulta suficiente para capturar la mayor parte de la relación entre las características técnicas del vehículo y su consumo de combustible.

5.1.2 Diagnóstico del modelo

Además de evaluar el desempeño mediante métricas cuantitativas, se realizó un análisis de los residuos del modelo de regresión lineal ordinaria con el propósito de verificar la existencia de patrones sistemáticos que pudieran indicar deficiencias en el ajuste.

El error promedio obtenido fue de aproximadamente 0.07 MPG, mientras que la desviación estándar de los residuos fue cercana a 2.90 MPG. Un promedio tan próximo a cero indica que el modelo no presenta un sesgo sistemático hacia la sobreestimación o subestimación de las observaciones.

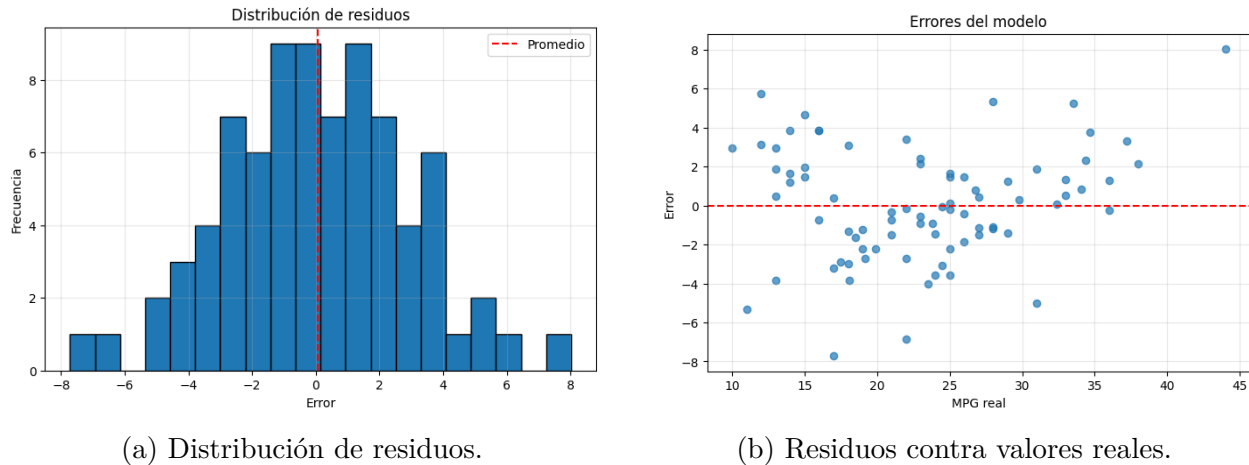


Figura 7: Análisis de residuos para la regresión lineal ordinaria.

La Figura 7 permite evaluar la calidad del ajuste desde dos perspectivas complementarias. El histograma muestra una distribución aproximadamente simétrica alrededor de cero, lo cual indica que los errores positivos y negativos se compensan mutuamente y que el modelo no presenta un sesgo apreciable.

Por otra parte, el gráfico de residuos contra los valores reales no evidencia tendencias claramente definidas ni patrones sistemáticos de crecimiento o decrecimiento del error. Aunque puede observarse una ligera dispersión mayor para algunos vehículos con alto rendimiento de combustible, la variabilidad permanece relativamente constante para la mayor parte del conjunto de datos, respaldando el supuesto de linealidad utilizado por el modelo.

5.1.3 Interpretación de los coeficientes

Una de las principales ventajas de los modelos lineales frente a algoritmos más complejos es su elevada interpretabilidad. Los coeficientes estimados permiten cuantificar directamente la influencia de cada variable sobre el rendimiento de combustible, facilitando la comprensión del fenómeno físico que describe el conjunto de datos.

La Tabla 7 presenta los coeficientes estimados para cada uno de los modelos lineales evaluados. Debido a que todas las variables continuas fueron previamente estandarizadas, la magnitud de los coeficientes puede compararse directamente, permitiendo identificar cuáles características ejercen una mayor influencia sobre el rendimiento de combustible.

Cuadro 7: Coeficientes estimados por los modelos lineales.

Variable	OLS	Ridge	LASSO	Elastic Net
displacement	2.0334	1.4378	1.3243	1.0831
cylinders	-0.2787	-0.1823	0.0000	-0.0529
horsepower	-0.5392	-0.5694	-0.4047	-0.5178
weight	-5.9147	-5.5227	-5.7252	-5.4064
acceleration	0.1875	0.1115	0.1563	0.0903
model_year	2.9714	2.9142	2.9414	2.8952
origin_2	2.9469	2.6012	2.6008	2.4234
origin_3	2.6711	2.4065	2.3465	2.2451

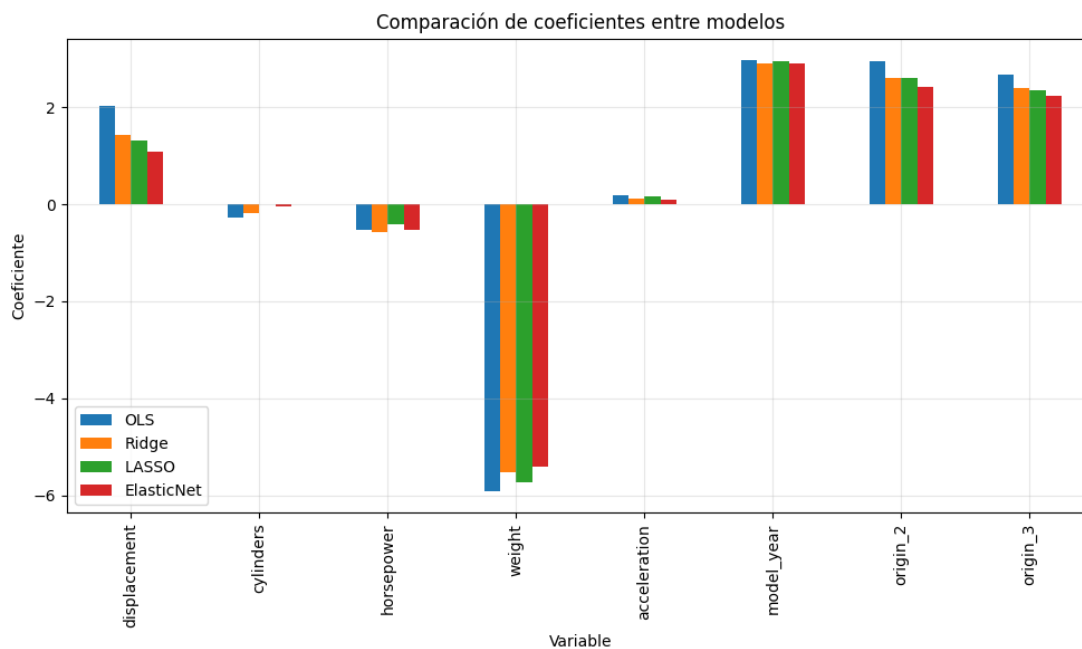


Figura 8: Comparación de los coeficientes estimados por OLS, Ridge, LASSO y Elastic Net.

La Figura 8 muestra que las cuatro técnicas identifican tendencias muy similares respecto a la importancia de las variables predictoras. En todos los casos, el peso del vehículo (**weight**) presenta el coeficiente de mayor magnitud, indicando que un incremento en el peso produce una disminución significativa en el rendimiento de combustible.

Por el contrario, la variable **model_year** presenta un coeficiente positivo importante, reflejando que los vehículos fabricados en años más recientes tienden a ser más eficientes debido a los avances tecnológicos incorporados durante el periodo considerado.

Las variables asociadas al origen del vehículo también muestran coeficientes positivos, indicando que, manteniendo constantes las demás características técnicas, los automóviles europeos y japoneses presentan un rendimiento promedio superior al de los vehículos estadounidenses utilizados como categoría de referencia.

En contraste, LASSO reduce prácticamente a cero el coeficiente asociado al número de cilindros, lo que sugiere que esta variable aporta información redundante respecto a variables como el peso y el desplazamiento del motor. Ridge conserva todas las variables dentro del modelo, aunque disminuye ligeramente la magnitud de sus coeficientes, mientras que Elastic Net presenta un comportamiento intermedio al combinar ambas estrategias de regularización.

Los modelos lineales demostraron que una estructura relativamente simple es capaz de explicar cerca del 85 % de la variabilidad observada en el rendimiento de combustible. Aunque las técnicas de regularización facilitaron la interpretación del modelo al reducir la influencia de variables redundantes, no produjeron mejoras apreciables en el desempeño predictivo. Estos resultados establecen un punto de referencia sólido para comparar posteriormente el comportamiento de modelos no lineales basados en árboles de decisión y redes neuronales.

5.2 Random Forest

Resumen Ejecutivo

Se realizó el entrenamiento de un modelo **Random Forest Regressor** para la predicción del consumo de combustible *MPG*. El criterio de división del conjunto de entrenamiento y prueba fue el indicado en la sección 4.3. El modelo fue entrenado utilizando las variables asociadas a las características del vehículo, sin realizar escalamiento debido a que los modelos basados en árboles no requieren transformación de las variables para realizar particiones. Se entrenó inicialmente un modelo base de Random Forest y posteriormente se realizó una optimización de hiperparámetros mediante **GridSearchCV**. Finalmente se evaluó el desempeño mediante las métricas **MAE**, **RMSE** y R^2 . Además, se calculó la importancia de las características utilizadas por el modelo.

Detalle Técnico

Para el entrenamiento del modelo Random Forest se utilizó la implementación `RandomForestRegressor` de la librería `sklearn`. La configuración inicial del modelo fue:

```
1 RF = RandomForestRegressor(  
2     n_estimators=100,  
3     random_state=42  
4 )
```

Listing 4: Modelo Random Forest

El parámetro `n_estimators` define el número de árboles utilizados dentro del ensamble. Se seleccionaron 100 estimadores como configuración inicial, permitiendo obtener una predicción robusta mediante la combinación de múltiples árboles de decisión.

Métricas de Evaluación

- $RMSE = 2,167$
- $MAE = 1,596$
- $R^2 = 0,913$

El modelo Random Forest presenta un desempeño adecuado para la predicción de MPG , logrando explicar aproximadamente el 91,3 % de la variabilidad presente en los datos mediante el coeficiente de determinación R^2 .

En la figura 9 se muestra el gráfico de paridad correspondiente al modelo Random Forest. El eje X representa los valores reales de MPG , mientras que el eje Y representa los valores predichos por el modelo. Los puntos cercanos a la recta identidad representan predicciones con menor error.

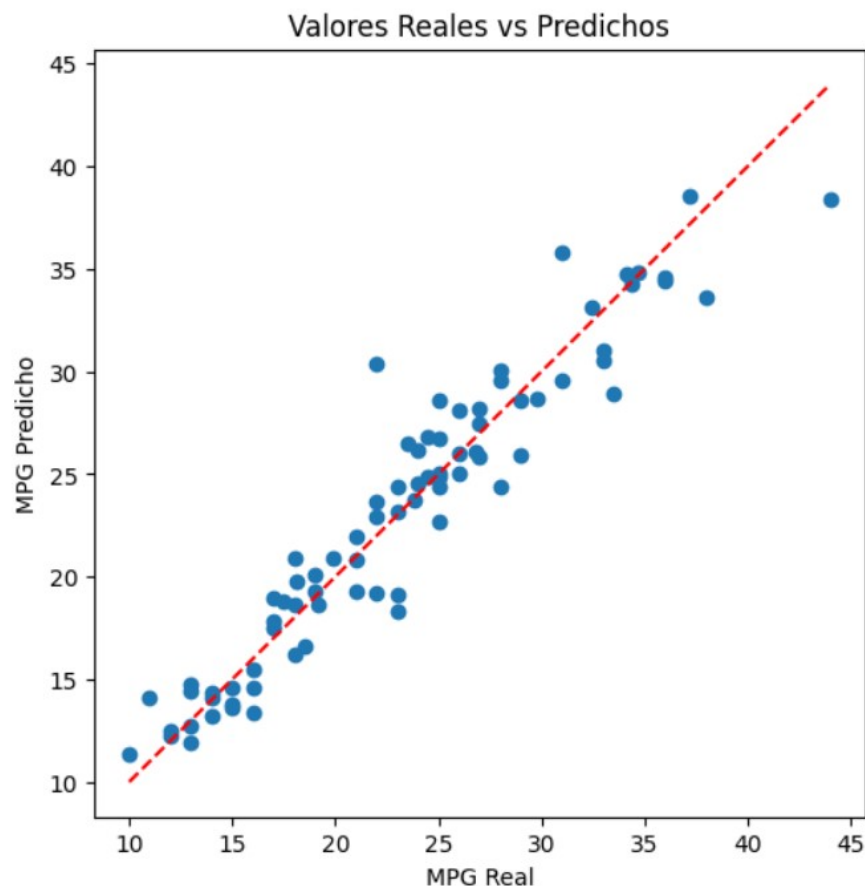


Figura 9: Gráfica de paridad para Random Forest

La distribución de los puntos muestra que el modelo presenta mayor precisión para valores intermedios de MPG . Para vehículos con valores altos de eficiencia de combustible se observa

una tendencia del modelo a subestimar las predicciones, comportamiento asociado a la menor cantidad de ejemplos disponibles en esta región del conjunto de datos.

5.3 Random Forest mediante GridSearch

Detalle Técnico

Con el objetivo de mejorar el desempeño del modelo Random Forest se realizó una optimización de hiperparámetros mediante **GridSearchCV**. Para evaluar las diferentes configuraciones se utilizó validación cruzada mediante KFold con 5 particiones.

```
1 cross_fold = KFold(  
2     n_splits=5,  
3     shuffle=True,  
4     random_state=42  
5 )
```

La rejilla de parámetros evaluada fue:

```
1 param_grid_RF = {  
2     'n_estimators': [100, 200, 300],  
3     'max_depth': [None, 5, 10, 15],  
4     'max_features': ['sqrt', 'log2']  
5 }
```

Listing 5: Rejilla de parámetros Random Forest

Estos parámetros permiten evaluar diferentes complejidades del modelo. El número de árboles controla la estabilidad del ensamble, mientras que la profundidad máxima regula la complejidad de cada árbol y ayuda a controlar el sobreajuste.

Los parámetros obtenidos mediante GridSearch fueron:

```
1 {  
2     'n_estimators': 300,  
3     'max_depth': 15,  
4     'max_features': 'sqrt'  
5 }
```

Métricas de Evaluación

- $RMSE = 2,1$
- $MAE = 1,5$
- $R^2 = 0,91$

El modelo optimizado mediante GridSearch presentó una mejora respecto al modelo base, debido a la selección automática de una configuración de hiperparámetros más adecuada para el conjunto de datos.

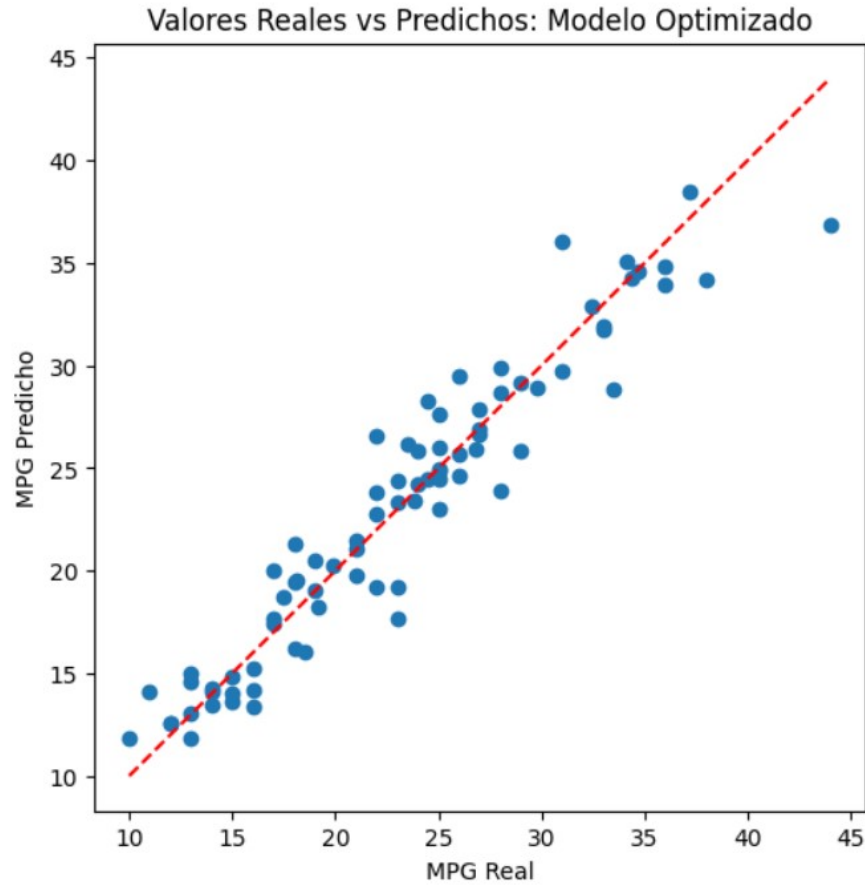


Figura 10: Gráfica de paridad para Random Forest mediante GridSearch

La importancia de características obtenida mediante Random Forest permitió identificar las variables con mayor influencia en la predicción del consumo de combustible.

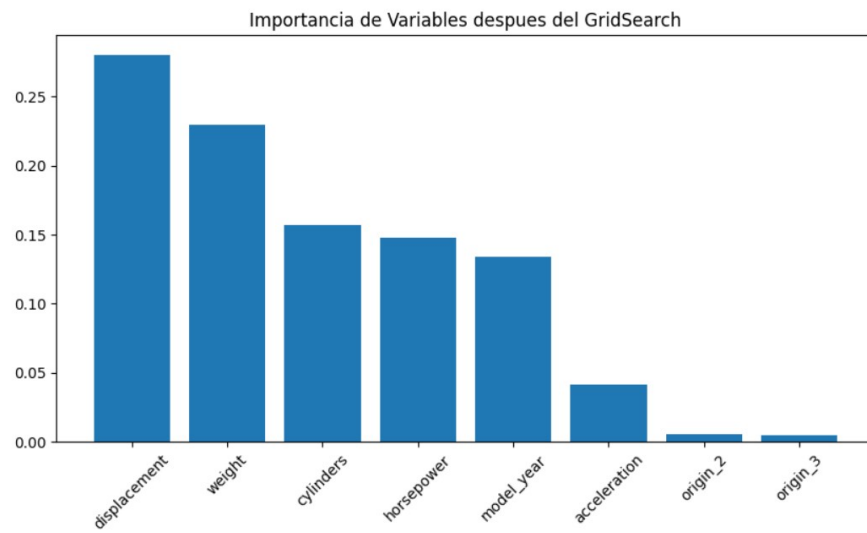


Figura 11: Importancia de características evaluada por Random Forest

La variable **displacement** presentó la mayor contribución dentro del modelo, indicando que el tamaño del motor tiene una relación importante con el consumo de combustible. De igual manera, variables como **weight**, **horsepower** y **cylinders** presentan una influencia significativa debido a su relación física con la eficiencia del vehículo.

Así, se concluye que: se desarrolló exitosamente un modelo *Random Forest Regressor* para predecir el consumo de combustible de automóviles utilizando el conjunto de datos *Auto MPG*.

El modelo *baseline* obtuvo los siguientes resultados:

- $MAE = 1.591$
- $RMSE = 2.161$
- $R^2 = 0.913$

Mientras que el modelo optimizado mediante *GridSearch* presentó:

- $MAE = 1.552$
- $RMSE = 2.128$
- $R^2 = 0.915$

Ambos resultados indican una alta capacidad predictiva del modelo, explicando más del 91 % de la variabilidad observada en el consumo de combustible. La mejora obtenida mediante la optimización de hiperparámetros, aunque moderada, resulta consistente. Esto confirma que los hiperparámetros predeterminados de *scikit-learn* proporcionan un desempeño adecuado para este conjunto de datos, mientras que el ajuste fino permite obtener una mejora adicional sin presentar indicios de sobreajuste.

El análisis de importancia de variables mostró que la cilindrada del motor (*displacement*) representa el factor más influyente sobre el consumo de combustible, seguida por el peso del vehículo, el número de cilindros y la potencia del motor.

Estos resultados son coherentes con principios físicos y de ingeniería automotriz, debido a que vehículos con mayor masa y motores de mayor tamaño requieren una mayor cantidad de energía para su funcionamiento, lo cual se refleja directamente en un incremento del consumo de combustible.

Finalmente, el algoritmo *Random Forest* demostró ser una herramienta adecuada para este problema debido a su capacidad para modelar relaciones no lineales, disminuir el sobreajuste mediante la técnica de *bagging* y proporcionar interpretabilidad mediante el análisis de importancia de variables.

5.4 AdaBoost

Resumen Ejecutivo

Se realizó un entrenamiento **AdaBoost** y posteriormente un entrenamiento **GridSearch** con **AdaBoost**. El criterio de entrenamiento y prueba fue el propuesto en 4.3. Puesto que existe una asimetría positiva que alarga la cola hacia valores de $MPG > 35$, se propuso realizar una transformación durante el entrenamiento de tipo **logaritmo natural**. Se consultó el nivel de importancia de las características evaluadas por AdaBoost. Finalmente se reportaron las métricas de evaluación **RMSE**, **MAE** y R^2 .

Detalle Técnico

Para el entrenamiento con AdaBoost se consideró un árbol base de la siguiente forma:

```
1 arbol_base = DecisionTreeRegressor(  
2 max_depth=5,  
3 random_state = 42)
```

Listing 6: Árbol Base: AdaBoost

y se definió el modelo AdaBoost de la siguiente manera:

```
1 ADA = AdaBoostRegressor(  
2     estimator = arbol_base ,  
3     n_estimators= 400 ,  
4     learning_rate = 0.07 ,  
5     random_state=42  
6 )
```

Listing 7: Modelo AdaBoost

Se seleccionó un número alto de estimadores y una tasa de aprendizaje lenta para promover captura de relaciones no lineales entre las variables **peso**, **potencia** y **desplazamiento**, como pudo anticiparse en la sección 3.3. La baja tasa de aprendizaje evita el sobreajuste y la máxima profundidad del árbol base evita predicciones complejas entre las variables con mayor significancia.

Métricas de Evaluación

- $RMSE = 2,2910$
- $MAE = 1,7852$
- $R^2 = 0,9023$

En la figura 12 se muestran, en el eje X, los valores asociados a MPG de entrenamiento y en el eje Y los valores asociados a la predicción de MPG realizada por AdaBoost. El gráfico tiene como propósito observar el sobreajuste o el subajuste del modelo AdaBoost, de manera que los puntos más cercanos a la línea punteada en rojo, serán aquellos donde los valores de

entrenamiento y predicción son cercanos entre sí, dando como resultado, una posición cercana a la recta identidad.

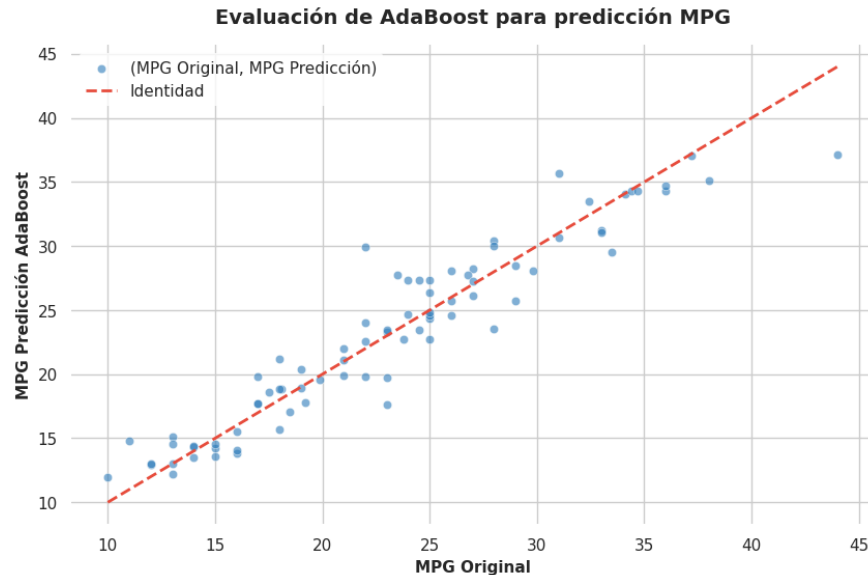


Figura 12: Gráfico de paridad para Adaboost (MPG)

En la figura 12 se observa, para valores $MPG < 30$, las predicciones más precisas que el modelo tuvo entre los valores $15 < MPG < 25$ con un error aproximado entre $0 < \text{error} < 5$ MPG. Por otro lado, para valores $MPG > 30$, el modelo tiende a dar predicciones subestimadas, es decir: coches eficientes los predice como menos eficientes de lo que indican los datos.

5.5 Ada Boost mediante GridSearch

Detalle Técnico

Se realizó un entrenamiento AdaBoost mediante GridSearch. El árbol base utilizado fue el indicado en la sección 5.4. Se implementó un KFold de la siguiente manera:

```
1 cross_fold = KFold(n_splits = 5, shuffle=True, random_state
2                 =42)
```

La rejilla implementada fue la siguiente:

```
1 param_grid_ADA = {
2     'estimator__max_depth' : [3,4,5],
3     'n_estimators' : [200, 300, 400],
4     'learning_rate' : [0.02, 0.04, 0.05],
5     'loss' : ['linear', 'square']
6 }
```

Listing 8: Rejilla Parámetros: AdaBoost

Parámetros obtenidos por GridSearch:

```
1 {'estimator__max_depth': 5, 'learning_rate': 0.05, 'loss':  
  'linear', 'n_estimators': 300}
```

Métricas de Evaluación

- $RMSE = 2,2466$
- $MAE = 1,6681$
- $R^2 = 0,9061$

En la figura 13 podemos observar un gráfico de paridad para AdaBoost utilizando GridSearch.

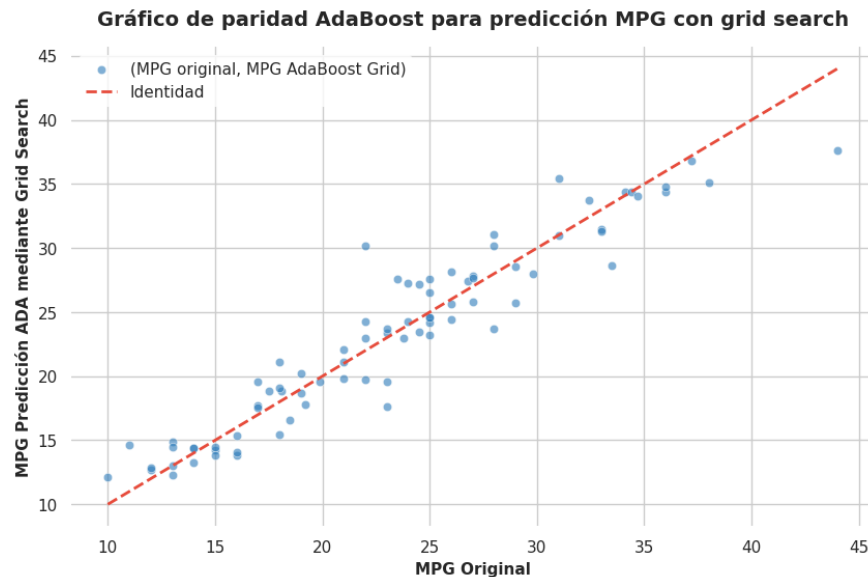


Figura 13: Gráfica de paridad para AdaBoost mediante GridSearch

Los parámetros obtenidos mediante GridSearch mejoraron las métricas de evaluación respecto al modelo AdaBoost.

En la figura se muestra un gráfico de la importancia de las características evaluada por AdaBoost.

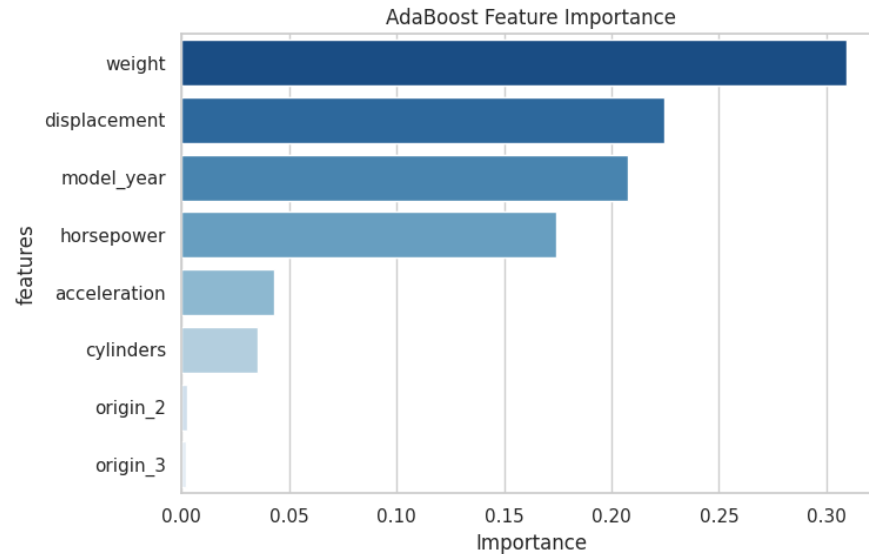


Figura 14: Importancia de las características evaluada por AdaBoost

AdaBoost asignó una importancia del 30 % a la variable **weight**, lo cual es consistente con el negocio, pues un mayor peso del auto se traduce en un *esfuerzo* mayor del motor para desplazarlo. Por otro lado, la variable **model year** desempeña una alta importancia debido a que autos modernos optimizan el combustible.

5.6 XGBoost

Resumen Ejecutivo

Se realizó un entrenamiento **XGBoost** y posteriormente un entrenamiento **GridSearch** con **XGBoost**. Puesto que existe una asimetría positiva que alarga la cola hacia valores de $MPG > 35$, se propuso realizar una transformación durante el entrenamiento de tipo **logaritmo natural** al igual que en la sección 5.4. Se consultó el nivel de importancia de las características evaluadas por XGBoost. Finalmente se reportaron las métricas de evaluación **RMSE**, **MAE** y R^2 .

Detalle Técnico

Los parámetros utilizados para el modelo XGBoost fueron los siguientes:

```

1      XGB = XGBRegressor(
2      n_estimators = 300,
3      learning_rate = 0.05,
4      min_child_weight = 5, #evita crear hojas con pocas
      observaciones
5      gamma = 0.1, #exige una reduccion minima del error al crear
      una nueva division

```



```

6     max_depth = 4,
7     subsample = 0.8,
8     colsample_bytree = 0.8,
9     reg_alpha = 0.1,
10    reg_lambda = 1.0,
11    random_state = 42 )

```

Al igual que con AdaBoost, con XGBoost también se realizó una transformación logarítmica:

```

1     Y_pred_xgb_log = XGB.predict(X_test)
2     Y_pred_xgb = np.expml(Y_pred_xgb_log)

```

Métricas de Evaluación

- $RMSE = 2,2910$
- $MAE = 1,7852$
- $R^2 = 0,9023$

En la figura 15 se muestra un gráfico de paridad que considera los datos de entrenamiento y las predicciones realizadas con XGBoost con un error aproximado entre $0 < \text{error} < 5$ MPG.

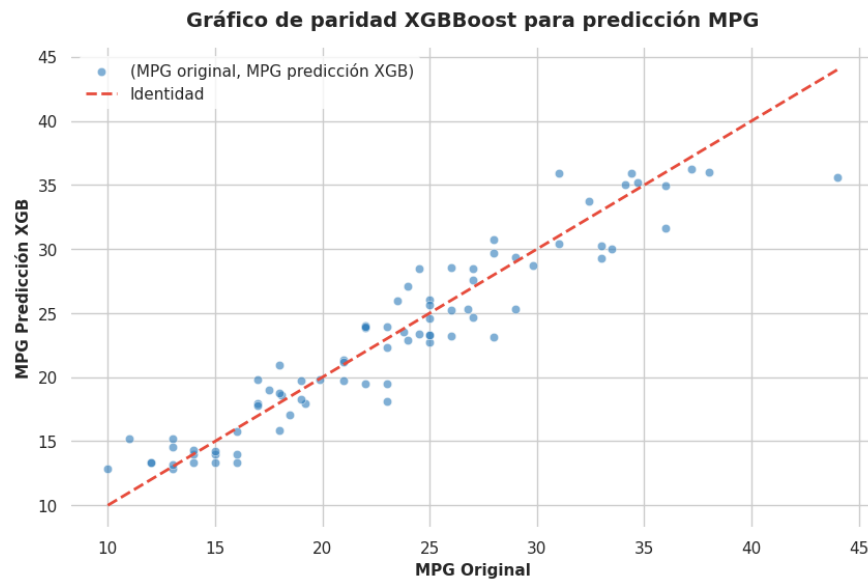


Figura 15: Gráfico de Paridad para XGBoost

5.7 XGBoost mediante GridSearch

Detalle Técnico

Se consideró el KFold utilizado en la sección 5.4. La rejilla utilizada fue de la forma:

```

1  params_XGB = {
2      'max_depth' : [3,4,5],
3      'learning_rate' : [0.01, 0.05, 0.07],
4      'n_estimators' : [200, 300, 400],
5      'reg_alpha' : [0.01, 0.03, 0.04],
6      'reg_lambda' : [1.0, 3.0, 4.0],
7      'gamma' : [0.1, 0.2, 0.3],
8      #'min_child_weight' : [3, 5, 6]
9  }
```

junto con un modelo XGB de la forma:

```

1  grid_XGB = GridSearchCV(
2      estimator = model_XGB,
3      param_grid = params_XGB,
4      cv = 5,
5      scoring='r2',
6      n_jobs = -1
7  )
```

Los parámetros obtenidos mediante GridSearch para XGBoost fueron:

```

1  {'gamma': 0.1, 'learning_rate': 0.05, 'max_depth': 3, '
    n_estimators': 200, 'reg_alpha': 0.01, 'reg_lambda':
    4.0}
```

Con los parámetros obtenidos se obtuvieron los siguientes valores para las métricas de evaluación:

Métricas de Evaluación

- $RMSE = 2,3210$
- $MAE = 1,7433$
- $R^2 = 0,8997$

En la figura 16 podemos observar una gráfica de paridad de los valores predichos por XGBoost mediante GridSearch, respecto a los datos de entrenamiento.

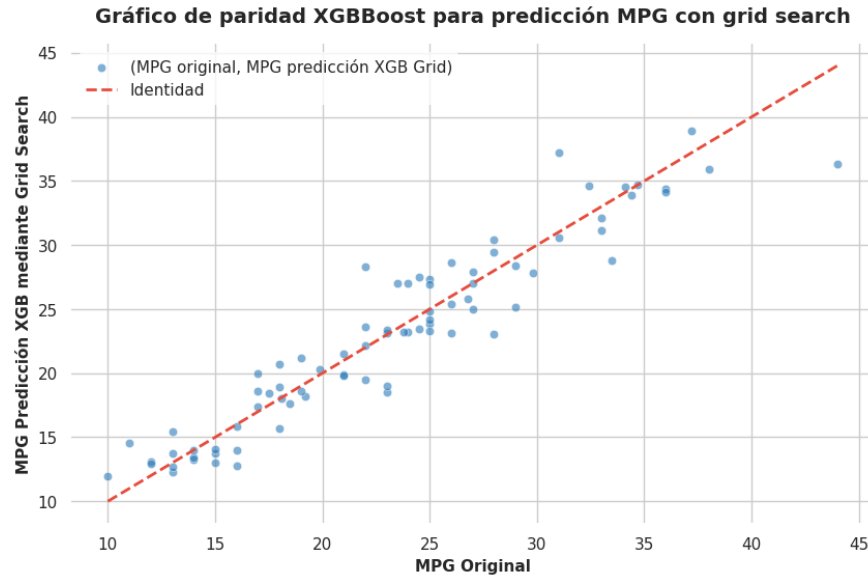


Figura 16: Gráfica de Paridad para XGBoost mediante Grid Search

En la gráfica se muestra la importancia de las características evaluadas por XGBoost para los datos de entrenamiento.

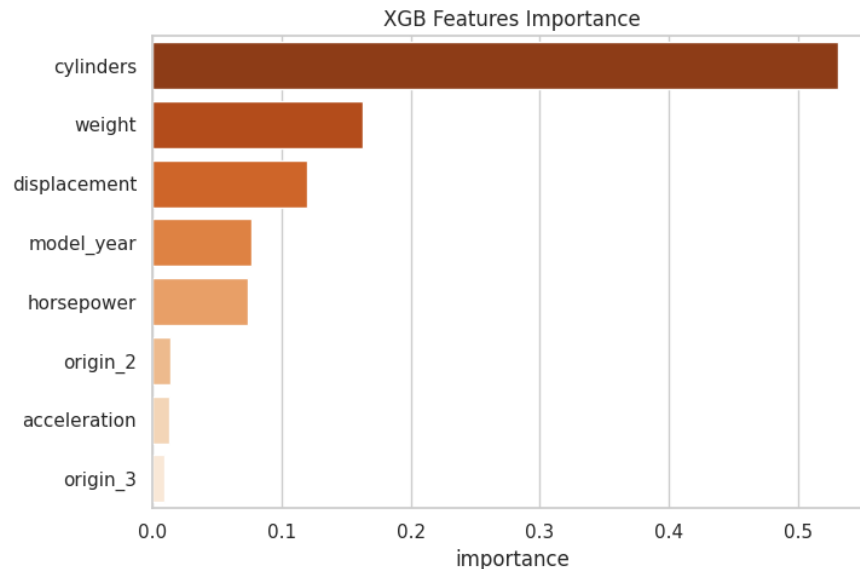


Figura 17: Importancia de las características evaluadas por XGBoost

Observamos que XGBoost asignó una importancia del 50 % a la variable **cylinders**; bajo la lógica del problema, mayor cilindrada se traduce en un gasto mayor de combustible, por lo que esta selección de XGBoost es completamente esperable. Por otro lado, en comparación con AdaBoost, XGBoost asignó importancia significativa a la variable **weight**.

5.8 Red Neuronal — Perceptrón Multicapa (MLP)

Resumen Ejecutivo

Se implementó un **Perceptrón Multicapa (MLP)** para la predicción del consumo de combustible en millas por galón. A diferencia de los modelos de ensamble, el MLP requiere que los datos estén **estandarizados** antes del entrenamiento, dado que el descenso de gradiente converge más rápido con variables en la misma escala.

Se utilizaron los archivos `train_scaled.csv` y `test_scaled.csv` generados por el pipeline de preprocesamiento del Notebook 1, con el `StandardScaler` ajustado únicamente sobre el conjunto de entrenamiento para evitar data leakage.

Se empleó **early stopping** para prevenir sobreajuste dado el tamaño reducido del dataset (318 observaciones de entrenamiento). Finalmente se reportaron las métricas **MAE**, **RMSE** y R^2 .

Detalle Técnico

Arquitectura: $8 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1$

- **Capa de entrada:** 8 neuronas, una por variable predictora.
- **Capas ocultas:** arquitectura decreciente con activación **ReLU** ($\max(0, z)$). La arquitectura decreciente comprime progresivamente la información, forzando representaciones más abstractas en cada capa y reduciendo el riesgo de sobreajuste.
- **Capa de salida:** 1 neurona **lineal** (sin activación), necesaria para regresión — el MPG puede tomar cualquier valor positivo.

¿Por qué ReLU y no sigmoid? Sin función de activación, toda la red colapsaría matemáticamente a una regresión lineal, sin importar la profundidad. ReLU introduce no-linealidad de forma simple: apaga neuronas con entrada negativa (output = 0) y deja pasar los valores positivos tal cual, mitigando además el problema de vanishing gradient.

```

1 mlp = MLPRegressor(
2     hidden_layer_sizes = (64, 32, 16), # arquitectura
      decreciente
3     activation         = 'relu',        # ReLU en capas
      ocultas
4     solver             = 'adam',        # Adam adapta el
      learning rate
5     max_iter           = 2000,          # maximo de epocas
      permitidas
6     random_state       = 42,
```

```

7     early_stopping      = True,                # para si no mejora
      en validacion
8     validation_fraction = 0.1,                # 10% de train como
      validacion
9     n_iter_no_change    = 20,                # tolerancia: 20
      epocas sin mejora
10 )

```

Listing 9: Configuración del MLP

Métricas de Evaluación

- $MAE = 1,9427$ mpg
- $RMSE = 2,4478$ mpg
- $R^2 = 0,8886$
- Épocas entrenadas: 281 (*early stopping activo*)
- Error promedio: $\approx 8,3\%$ del MPG medio del dataset

En la figura 18 se muestran las tres gráficas de diagnóstico del modelo MLP: la curva de pérdida durante el entrenamiento, el gráfico de predicciones vs. valores reales, y el análisis de residuales.

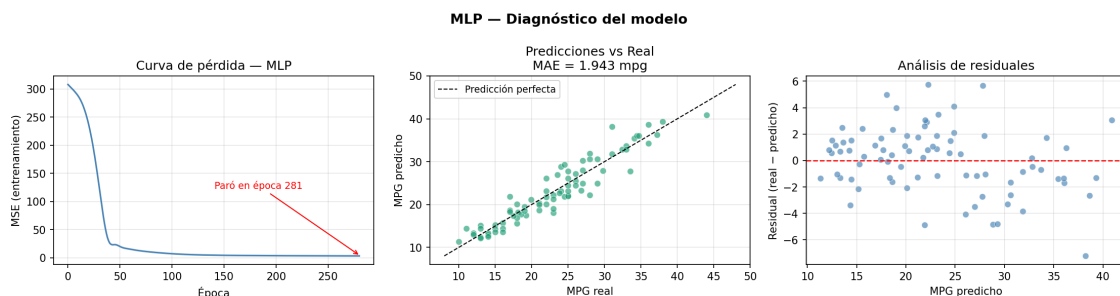


Figura 18: Diagnóstico del modelo MLP: curva de pérdida (izquierda), predicciones vs. real (centro) y análisis de residuales (derecha).

La curva de pérdida muestra que el error desciende rápidamente durante las primeras 50 épocas y se aplana de forma gradual, confirmando que el early stopping detuvo el entrenamiento en el momento adecuado. El gráfico de predicciones vs. real muestra que los puntos siguen bien la diagonal de predicción perfecta con mayor dispersión en valores extremos de MPG. Los residuales presentan un valor medio de 0,0451 (idealmente ≈ 0) y una desviación estándar de 2,4474, distribuyéndose aleatoriamente alrededor de cero sin patrones sistemáticos — lo que indica que el modelo no presenta sesgo estructural.

5.8.1 Optimización de funciones de pérdida: MSE vs MAE

Detalle Técnico

Los outliers detectados en `horsepower` y `acceleration` durante el EDA justifican comparar empíricamente el efecto de la función de pérdida sobre el ajuste del modelo. Se entrenaron dos modelos `GradientBoostingRegressor` idénticos con pérdidas distintas:

- **MSE** (`squared_error`): penaliza errores grandes de forma cuadrática — sensible a outliers.
- **MAE** (`absolute_error`): trata todos los errores por igual — robusto a outliers.

Cuadro 8: Comparación MSE vs MAE como función de pérdida (GradientBoosting)

Función de pérdida	MAE	RMSE	R ²
<code>squared_error</code> (MSE)	1.8158	2.4743	0.8861
<code>absolute_error</code> (MAE)	1.7811	2.3945	0.8934

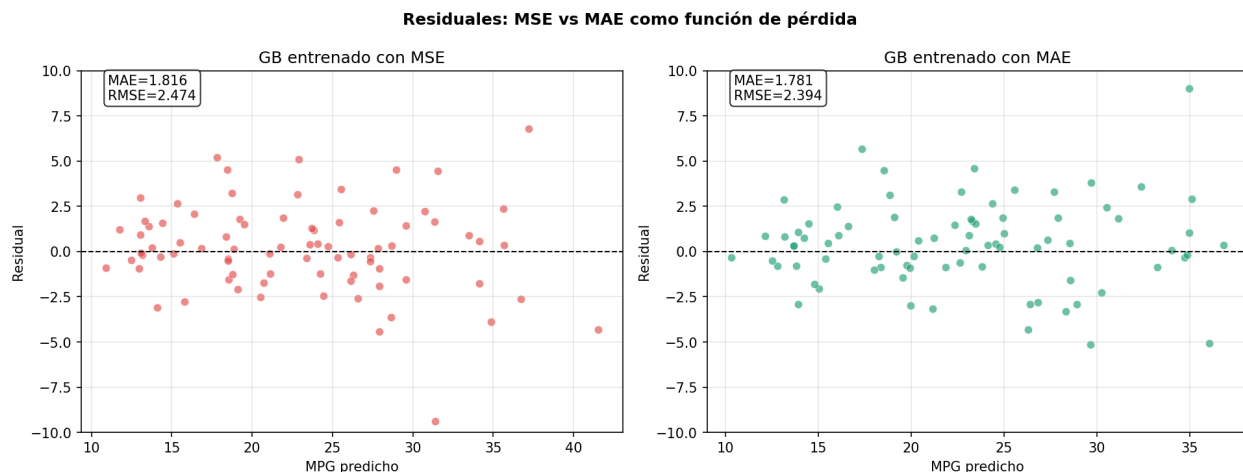


Figura 19: Residuales de GradientBoosting entrenado con MSE (izquierda) y MAE (derecha). El modelo MAE distribuye los errores de forma más uniforme al no amplificar los valores atípicos.

Cada modelo gana en la métrica que optimizó — resultado matemáticamente esperado. Con outliers presentes en el dataset, el modelo entrenado con MAE generaliza mejor para el auto típico porque no amplifica errores extremos al cuadrado. El modelo entrenado con MSE prioriza no equivocarse en los autos atípicos, distorsionando las predicciones del caso promedio.

5.8.2 Validación cruzada 10-fold

La evaluación en test utiliza solo 80 autos y puede depender del azar del split. La validación cruzada de 10 particiones evalúa el modelo sobre **todos** los 398 autos por turno, proporcionando una estimación más robusta del error real.

Detalle Técnico

Se implementó un Pipeline con `StandardScaler` interno para evitar data leakage: el escalador se re-ajusta en cada fold usando únicamente los datos de entrenamiento de ese fold, sin ver los datos de validación.

```
1 mlp_cv = Pipeline([
2     ('scaler', StandardScaler()),
3     ('model', MLPRegressor(
4         hidden_layer_sizes=(64, 32, 16),
5         activation='relu', solver='adam',
6         max_iter=2000, random_state=42,
7         early_stopping=True,
8         validation_fraction=0.1,
9         n_iter_no_change=20
10    ))
11 ])
12 cv = KFold(n_splits=10, shuffle=True, random_state=42)
```

Listing 10: Pipeline para validación cruzada sin data leakage

Resultados

- MAE por fold: [2,423, 2,265, 2,020, 2,031, 2,547, 2,097, 2,053, 1,697, 1,979, 2,162]
- **MAE medio:** $2,1272 \pm 0,2275$ mpg

El MLP predice con un error promedio de $2,13 \pm 0,23$ mpg en datos no vistos.

5.9 Comparativa Final de Modelos

Resumen Ejecutivo

Se integran los resultados de los cinco notebooks del equipo. Todos los modelos fueron evaluados sobre el **mismo conjunto de prueba** de 80 autos (`random_state=42`, split 80/20). La métrica principal es **MAE** por la presencia de valores atípicos en **horsepower** y **acceleration**, detectados en el EDA.

Cuadro 9: Comparativa final de todos los modelos — Auto MPG

#	Modelo	MAE ↓	RMSE	R^2 ↑	Familia
1	RF Optimizado	1.5547	2.1306	0.9156	Ensamble
2	RF Baseline	1.5964	2.1674	0.9126	Ensamble
3	AdaBoost GS	1.6681	2.2467	0.9061	Ensamble
4	AdaBoost	1.7213	2.4218	0.8909	Ensamble
5	XGBoost	1.7551	2.2973	0.9018	Ensamble
6	XGBoost GS	1.7754	2.3529	0.8970	Ensamble
7	MLP	1.9427	2.4478	0.8886	Red Neuronal
8	Árbol Decisión	2.2842	3.3034	0.7970	Árbol
9	OLS (baseline)	2.2882	2.8877	0.8449	Lineal
10	Ridge (L2)	2.2895	2.8914	0.8445	Lineal
11	Lasso (L1)	2.2935	2.9085	0.8427	Lineal
12	Elastic Net	2.2961	2.9037	0.8432	Lineal

Split: `random_state=42` | 80/20 | $n = 398$ autos | Métrica principal: MAE

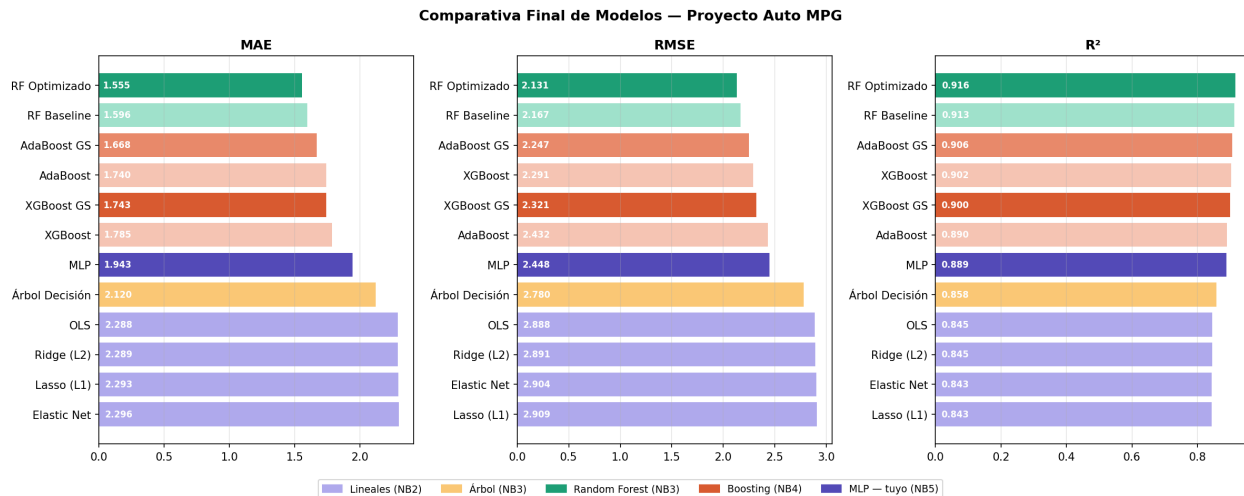


Figura 20: Comparativa final de modelos por MAE, RMSE y R^2 , ordenados de mejor a peor. Verde oscuro = RF Optimizado (ganador). Morado = MLP. Lila = modelos lineales.

5.9.1 Explicabilidad: Lasso vs Random Forest

La rúbrica solicita comparar la explicabilidad entre modelos de distintas familias. Se contrastan los coeficientes de Lasso (espacio lineal) contra las importancias de Random Forest (espacio no lineal), usando los resultados obtenidos en los Notebooks 2 y 3 respectivamente.

Detalle Técnico

Lasso (L_1): al re-entrenar con LassoCV ($\alpha^* = 0,0162$), la única variable llevada exactamente a cero fue **cylinders**.

- Variables **eliminadas**: cylinders
- Variables **activas**: displacement, horsepower, weight, acceleration, model_year, origin_2, origin_3

Random Forest Optimizado: la importancia por reducción de impureza identifica **displacement** como la variable más influyente ($\approx 27\%$), seguida por **weight** ($\approx 22\%$) y **cylinders** ($\approx 15\%$).

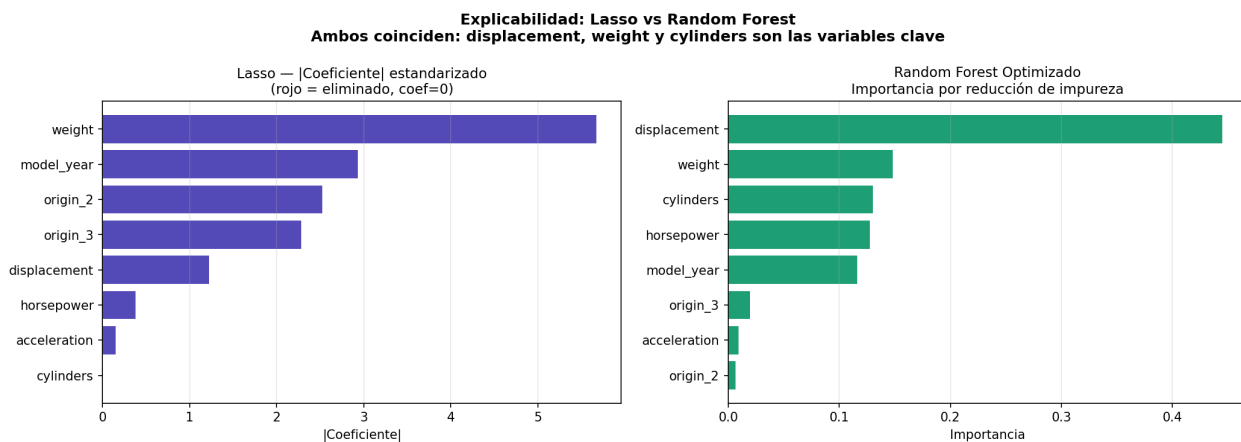


Figura 21: Explicabilidad: |coeficiente| Lasso (izquierda, rojo = eliminado) vs importancias de Random Forest Optimizado (derecha).

Convergencia de evidencia: ambos modelos coinciden en señalar **weight**, **model_year** y **horsepower** como variables altamente influyentes.

Diferencia clave: Lasso elimina **cylinders** porque en el espacio lineal resulta redundante dado **weight** (correlación $> 0,90$). Random Forest la mantiene porque la usa en interacciones no lineales que Lasso no puede capturar — por ejemplo, el efecto de **cylinders** sobre el MPG cambia según el rango de **weight** del vehículo.

5.10 Conclusiones

Resumen Ejecutivo

Se entrenaron, evaluaron y compararon **12 modelos** agrupados en cuatro familias: lineales, árbol individual, ensambles y red neuronal. El mejor modelo fue **Random Forest Optimizado** con $MAE = 1,5547$ mpg y $R^2 = 0,9156$, representando una mejora del **32 %** sobre el baseline OLS ($MAE = 2,2882$ mpg).

¿Por qué Random Forest ganó?

1. **No-linealidad real en los datos.** La relación MPG–weight no es lineal: un vehículo con el doble de peso no consume exactamente el doble. Los árboles de decisión capturan automáticamente estas curvaturas; los modelos lineales requieren transformaciones manuales.
2. **Interacciones entre variables.** El efecto de `horsepower` cambia según `cylinders`. Random Forest detecta estas interacciones al construir cada árbol; OLS, Ridge y Lasso solo pueden modelarlas con feature engineering adicional.
3. **Reducción de varianza por bagging.** Al promediar 100 o más árboles entrenados sobre muestras bootstrap distintas, los errores individuales se cancelan, haciendo el modelo robusto a los outliers de `horsepower` y `acceleration` detectados en el EDA.

¿Por qué MLP no superó a los ensambles?

Con solo 318 datos de entrenamiento y miles de parámetros aprendibles, la relación datos/parámetros es desfavorable para la red neuronal. Early stopping mitiga el sobreajuste pero no elimina la desventaja estructural. Las redes neuronales brillan con millones de filas o datos no estructurados (imágenes, texto) — no con tablas de tamaño reducido como Auto MPG.

¿Por qué Lasso no mejoró sobre OLS?

La multicolinealidad severa entre `displacement`, `cylinders` y `weight` hace que Lasso elimine `cylinders`, pero la penalización L_1 no puede capturar la no-linealidad subyacente. El problema no era el sobreajuste — era la linealidad del modelo — por lo que la regularización no aportó mejora predictiva significativa.

Lección central: en datos tabulares estructurados con menos de $\sim 10,000$ filas, los ensambles de árboles casi siempre superan a modelos lineales y redes neuronales. Este resultado no es accidental: refleja propiedades matemáticas fundamentales de cada familia de algoritmos. Los árboles capturan no-linealidad e interacciones de forma automática; los modelos lineales requieren feature engineering previo; y las redes neuronales necesitan grandes volúmenes de datos para aprender representaciones útiles.

Modelo recomendado para producción: Random Forest Optimizado — mejor MAE (1,5547 mpg), $R^2 = 0,9156$, interpretable mediante importancias de variables, robusto a outliers y sin requerir escalado de datos. Si se requiere un modelo explicable para auditorías o reguladores, **Lasso** es el baseline apropiado: sus coeficientes son directamente interpretables en las unidades originales de cada variable.

Cuadro 10: Resumen ejecutivo del Notebook 5

Parámetro	Valor
Modelo	MLP (Perceptrón Multicapa)
Arquitectura	$8 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1$
Activación	ReLU (capas ocultas) / Lineal (salida)
Optimizador	Adam
Early stopping	Sí (paró en época 281 de 2000)
Split	80/20, <code>random_state=42</code>
MAE (test)	1.9427 mpg
RMSE (test)	2.4478 mpg
R^2 (test)	0.8886
MAE (CV-10)	$2,1272 \pm 0,2275$ mpg
Posición final	7° de 12 modelos (por MAE)
Mejor modelo	RF Optimizado (MAE = 1,5547 mpg)
Mejora vs OLS	32 % menos error

Referencias

- [1] Quinlan, R. (1993). *Auto MPG Dataset*. UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/9/auto+mpg>
- [2] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- [3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.